Health-Check Software für die Dienste der Firma

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:
Elias Brandstätter
Zoe Emily Öllinger

Betreuer:
Prof. DI Christian Aberger

Projektpartner:
hello again GmbH

Leonding, 7. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 7. April 2026 Elias Brandstätter & Zoe Emily Öllinger

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Ausgangslage 1

1.2 Problemstellung 1

1.3 Ziel 1

2 Umfeldanalyse **3**

3 Technologien **4**

3.1 Overview 4

3.2 Frontend 4

3.3 Backend 7

3.4 Cloud & Datenerhaltung 11

3.5 Externe Schnittstellen 15

4 Umsetzung **19**

4.1 UI und UX 19

4.2 Dashboard 24

4.3 Systemarchitektur 29

4.4 Backend-Implementierung 34

4.5 Integration externer Systeme 34

4.6 Datenmanagement 34

4.7 Fehlerbehandlung und Logging 34

4.8 Sicherheit und Zugriff 34

5 Zusammenfassung **37**

Literaturverzeichnis **VI**

Abbildungsverzeichnis **VIII**

Tabellenverzeichnis **IX**

Quellcodeverzeichnis **X**

Anhang **XI**

1 Einleitung

1.1 Ausgangslage

Hello Again ist ein Unternehmen, das sich auf die Entwicklung von Kundenbindungs-Apps für kleine und mittelständische Unternehmen spezialisiert hat. Diese Apps werden über verschiedene Apple- und Google-Developer-Accounts veröffentlicht und gepflegt, wobei je nach Kunde und Plattform unterschiedliche Accounts und Konfigurationen zum Einsatz kommen. Da die Anzahl der betreuten Kunden-Apps kontinuierlich wächst, steigt damit auch der Aufwand, den Status und die Korrektheit all dieser Apps und der zugehörigen Accounts im Blick zu behalten.

1.2 Problemstellung

In der Praxis zeigt sich, dass Probleme mit einzelnen Kunden-Apps häufig nicht sofort erkannt werden. Typische Ursachen sind fehlende bzw. abgelaufene Verifizierungen oder fehlerhafte Konfigurationen in den Developer Accounts. Solche Probleme können dazu führen, dass eine App eingeschränkt verfügbar ist oder im schlimmsten Fall aus dem jeweiligen Store entfernt wird - mit direkten Konsequenzen für den betroffenen Kunden.

Da diese Informationen bislang über mehrere Plattformen verteilt sind und keine zentrale Übersicht existiert, ist eine systematische Überwachung aller Apps mit vertretbarem Aufwand kaum möglich. Probleme werden dadurch oft erst dann bemerkt, wenn sie bereits eskaliert sind und der Handlungsdruck hoch ist. Ein proaktiver, strukturierter Ansatz zur Fehlererkennung fehlt.

1.3 Ziel

Ziel dieses Projekts ist die Entwicklung eines zentralen Health-Monitoring-Dashboards, das den Status aller betreuten Kunden-Apps auf einen Blick erfassbar macht. Das

System aggregiert Informationen aus verschiedenen Quellen - darunter die Google Play Developer API, die App Store Connect API, HubSpot sowie Asana - und stellt diese in einem übersichtlichen Dashboard dar.

Jede App wird dabei automatisch einer von drei Statuskategorien zugeordnet: *Healthy*, *Needs Attention* oder *Critical*. Diese Kategorisierung ermöglicht es, den Handlungsbedarf auf einen Blick einzuschätzen und Prioritäten gezielt zu setzen. Darüber hinaus unterstützt das System den Fehlerbehebungsprozess aktiv, indem es strukturierte Hinweise auf die Ursache eines Problems liefert und direkte Maßnahmen vorschlägt.

Um den Arbeitsalltag weiter zu erleichtern, bietet das Dashboard umfangreiche Filter- und Suchmöglichkeiten, mit denen sich Apps nach bestimmten Fehlermeldungen, Parametern oder Plattformen eingrenzen lassen. Insgesamt soll das System dazu beitragen, Probleme frühzeitig zu erkennen, Reaktionszeiten zu verkürzen und die Qualität der betreuten Apps nachhaltig zu verbessern.

2 Umfeldanalyse

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula. Citing [1] properly.

Was ist eine GUID? Eine GUID kollidiert nicht gerne.

Kabellose Technologien sind in abgelegenen Gebieten wichtig [2].

3 Technologien

3.1 Overview

Allgemeines und Bild über den Tech-Stack...

3.2 Frontend

3.2.1 Vue.js

Das Frontend des Health-Monitoring-Dashboards wurde mit dem JavaScript-Framework Vue.js implementiert. Die Entscheidung für Vue.js basiert auf seiner einfachen Integration, klaren Struktur sowie der Weiterentwicklung innerhalb der Firma. Da innerhalb der Firma für die Entwicklung des Dashboards bereits genutzt wird, bietet uns diese Technologie perfekte Vorraussetzungen um implementiert zu werden. Dabei bietet Vue.js uns auch eine komponentenbasierte Architektur, wodurch einzelne UI-Bestandteile unabhängig voneinander Entwickelt und wiederverwendet werden können. Hierbei wurden uns von der Firma bereits fertige Komponenten zur Verfügung gestellt, wie zum Beispiel die Pagination. Für uns sind die Komponenten von Vorteil, da wir im Dashboard Übersichtsseiten, Detailseiten und Statusanzeigen nutzen. Dabei muss man beachten, dass von cer Firma für das Frontend zwei Komponenten gewünscht waren. Einmal für die Übersicht und einmal für die Detailseite.

Ein weiterer Vorteil bei unserer Verwendung von Vue.js ist, die reaktive Datenbindung, da Änderungen an den Daten automatisch im User Interface Aktualisiert werden, wodurch eine dynamische und aktuelle Darstellung der Health-Daten gewährleitstet wird.

Framework Background

Vue.js wurde von Evan You entwickelt und erstmals im Jahr 2014 veröffentlicht. Hierbei verfolgt das Framework einen progressiven Ansatz, bei dem die Entwickler selbst entscheiden können, wie umfangreich man Vue in sein Projekt integriert.

Ein zentrales Konzept von Vue ist das "Reactive Data Binding". Dabei wird das DOM automatisch aktualisiert, wenn sich Daten verändern.

Zusätzlich bietet Vue eine klare, strukturierte Trennung zwischen Template, Styling und Logik, wodurch die Wartbarkeit und Erweiterbarkeit der Anwendung besser verständlich ist.

Einsatz im Frontend

Im Rahmen dieses Projekts übernimmt Vue.js folgende zentrale Aufgaben:

- Darstellung der Health-Check-Daten
- Visualisierung von Statusinformationen und Fehlermeldungen
- Interaktion mit den APIS des Backends
- Dynamisches Laden und Aktualisieren von Daten
- Navigation zwischen verschiedenen Ansichten

3.2.2 UI&UX Design mit Figma

Für die Gestaltung der Benutzeroberfläche wurde das Design-Tool Figma verwendet. Figma ermöglicht eine kollaborative Erstellung von UI-Designs und bietet uns umfangreiche Funktionen zur Visualisierung unserer Benutzeroberflächen.

In unserem Projekt diente Figma als zentrales Werkzeug zur Planung uns Ausarbeitung des User Interfaces und des gewünschten Workflows. Dabei wurden von uns Layouts, Workflow und Farbkonzepte entwickelt, bevor diese dann technisch umgesetzt wurden.

Designprozess

Der Designprozess erfolgte iterativ und war stark durch regelmäßige Feedback-Stunden geprägt. In mehreren Workfloe- und Feedback-Sessions wurden die Entwürfe gemeinsam

besprochen und anschließend verbessert und an die Anforderungen der Mitarbeiter angepasst.

Diese Vorgehensweise ermöglichte es uns, frühzeitig Verbesserungen zu erkennen und entsprechend darauf zu reagieren und die Benutzerfreundlichkeit auf Wunsch der Mitarbeiter kontinuierlich zu optimieren.

Vorteile von Figma

Der Einsatz von Figma bringt mehrere Vorteile mit sich:

- Zentrale und kollaborative Designplattform
- Schnelle Erstellung von Prototypen
- Einfache Abstimmung im Team
- Möglichkeit zur direkten Visualisierung von User Flows
- Konsistente Gestaltung durch wiederverwendbare Komponenten

3.2.3 Benutzerfreundlichkeit und Interaktion

Ein zentrales Ziel des Frontends war die Entwicklung einer intuitiven und übersichtlichen Benutzeroberflache.

Besonderes Augenmerk wurde auf folgende Aspekte gelegt:

- Übersichtliche Darstellung von Statusinformationen
- Klare visuelle Hervorhebung von Fehlern und Warnungen
- Schnelle Navigation zwischen verschiedenen Bereichen
- Minimierung von Ladezeiten durch effiziente Datenabfragen
- Kommentarsektion zur Plattform-Übergreifenden bearbeitung
- Anzeige von gravierenden Fehlern mit Verlinkung zur Fehlerbehebung und Dokumentation

3.2.4 Vue.js

3.2.5 ...

3.3 Backend

3.3.1 Python

Das Backend des Health-Monitoring-Systems wurde mit der Programmiersprache Python implementiert. Die Entscheidung für Python war dabei aus mehreren Gründen naheliegend: Einerseits entspricht Python den internen Entwicklungsrichtlinien des Unternehmens, andererseits zeichnet sich die Sprache durch eine hohe Lesbarkeit und klare Syntax aus, die es ermöglicht, auch komplexe Logik übersichtlich zu strukturieren.

Darüber hinaus verfügt Python über ein umfangreiches Ökosystem an Bibliotheken für Webentwicklung, Datenanalyse und Cloud-Integration. Gerade im Kontext dieses Projekts ist die Verfügbarkeit offizieller Clientbibliotheken für Google BigQuery von besonderer Bedeutung, da diese die Anbindung an die Datenbankinfrastruktur erheblich vereinfachen. Nicht zuletzt profitiert das Projekt von der starken Community rund um Python, die umfangreiche Dokumentationen und etablierte Best Practices bereitstellt - beides Faktoren, die die langfristige Wartbarkeit des Systems begünstigen.

Hintergrund der Sprache

Python ist eine interpretierte, objektorientierte Programmiersprache, die ursprünglich von Guido van Rossum entwickelt und erstmals 1991 veröffentlicht wurde. Das erklärte Ziel bei ihrer Entwicklung war, eine Sprache zu schaffen, die gleichermaßen einfach zu lesen und effizient zu verwenden ist. Erreicht wird das durch eine minimalistische Syntax, die sich stark an natürlicher Sprache orientiert und dadurch die Lesbarkeit und Nachvollziehbarkeit von Code deutlich verbessert.

Im Gegensatz zu kompilierten Sprachen wird Python interpretiert ausgeführt, was die Entwicklung beschleunigt, da Änderungen am Code unmittelbar getestet werden können. Für die Entwicklung von Web-Backends und Datenverarbeitungssystemen ist diese Eigenschaft besonders vorteilhaft. [3, 4]

Einsatz im Backend

Im Rahmen dieses Projekts übernimmt Python mehrere zentrale Aufgaben innerhalb der Backend-Architektur: die Implementierung der REST-API, die Kommunikation mit externen Schnittstellen, die Verarbeitung und Aggregation von Monitoring-Daten, Datenbankabfragen über BigQuery sowie die Logik zur Statusanalyse und Fehlererkennung.

Besonders hervorzuheben ist dabei die Fähigkeit, verschiedene externe Systeme über API-Schnittstellen anzubinden. Im Health-Monitoring-System werden Daten aus mehreren unterschiedlichen Plattformen verarbeitet - darunter App-Store-Systeme, CRM-Systeme und Projektmanagement-Tools. Python ermöglicht es, diese heterogenen Datenquellen in einer zentralen Backend-Logik zusammenzuführen und strukturiert an das Frontend weiterzugeben.

3.3.2 FastAPI

Für die Implementierung der Backend-Schnittstellen wurde das Python-Framework FastAPI eingesetzt. FastAPI ist ein modernes, leistungsfähiges Framework zur Entwicklung von Web-APIs, das auf dem ASGI-Standard (Asynchronous Server Gateway Interface) basiert und eine schnelle sowie strukturierte Entwicklung von REST-Schnittstellen ermöglicht. Im Health-Monitoring-System stellt FastAPI die Endpunkte bereit, über die das Frontend Statusinformationen zu den überwachten Apps abruft und externe APIs angebunden werden.

Hintergrund des Frameworks

FastAPI wurde von Sebastián Ramírez entwickelt und 2018 erstmals veröffentlicht. Das Framework macht konsequenten Gebrauch von modernen Python-Features - insbesondere Typannotationen und asynchroner Programmierung - und hebt sich dadurch von älteren Frameworks wie Flask oder Django REST Framework ab.

Ein zentrales Merkmal von FastAPI ist die Nutzung von Python-Typdefinitionen zur automatischen Validierung von Anfragen und Antworten. Request-Parameter und Response-Objekte können dadurch klar definiert werden, was die Stabilität und Vorhersehbarkeit der API erhöht. Zusätzlich generiert FastAPI automatisch eine OpenAPI-

Spezifikation samt interaktiver Dokumentation, die Entwicklern direkt im Browser zur Verfügung steht und das Testen einzelner Endpunkte erheblich erleichtert. [5]

REST-API-Architektur

Das Backend des Health-Monitoring-Systems folgt einer klassischen REST-Architektur, bei der das System verschiedene Endpunkte bereitstellt, über die Daten abgefragt oder Aktionen ausgelöst werden können. Dazu zählen etwa das Abrufen aller Health-Checks, das Anzeigen von Details zu einem bestimmten Kunden, das Erstellen von Tasks oder Kommentaren sowie das Abfragen von Ticket-Informationen. Diese Endpunkte werden in FastAPI über Route-Deklarationen definiert, wobei jede Route einer spezifischen HTTP-Methode - etwa GET oder POST - zugeordnet ist.

Die REST-Architektur ermöglicht eine klare Trennung zwischen Frontend und Backend, sodass beide Systeme unabhängig voneinander weiterentwickelt werden können, solange die vereinbarte Schnittstellendefinition eingehalten wird.

Performance und Skalierbarkeit

FastAPI gehört zu den performantesten Python-Webframeworks, was sich vor allem auf die native Unterstützung asynchroner Programmierung zurückführen lässt. [6] Im Kontext dieses Projekts ist das besonders relevant, da im Zuge einer einzelnen Anfrage häufig mehrere externe APIs parallel abgefragt werden müssen. Durch die asynchrone Verarbeitung können diese Anfragen gleichzeitig ausgeführt werden, was die Gesamtantwortzeit spürbar reduziert und das System auch bei höherer Last stabil hält.

3.3.3 Uvicorn

Zur Ausführung der FastAPI-Anwendung wird der ASGI-Server Uvicorn eingesetzt. Uvicorn ist ein leichtgewichtiger, performanter Webserver für Python-Anwendungen und übernimmt die Aufgabe, eingehende HTTP-Requests entgegenzunehmen und an die FastAPI-Anwendung weiterzuleiten. Nach der Verarbeitung durch das Framework wird die generierte Response über Uvicorn an den Client zurückgesendet. [7]

Der ASGI-Standard

Uvicorn basiert auf dem ASGI-Standard (Asynchronous Server Gateway Interface), der eine moderne Alternative zum älteren WSGI-Standard darstellt. Der wesentliche Unterschied liegt in der Unterstützung asynchroner Verarbeitung: Während bei klassischen synchronen Webservern jede eingehende Anfrage einen eigenen Thread blockiert, können ASGI-basierte Server mehrere Operationen gleichzeitig verwalten, ohne dabei auf den Abschluss einer einzelnen Anfrage warten zu müssen. Das ist besonders vorteilhaft für Anwendungen, die häufig externe APIs aufrufen oder auf Datenbankabfragen warten - beides trifft auf das Health-Monitoring-System zu. [8]

Einsatz im Projekt

Im Health-Monitoring-System fungiert Uvicorn als Laufzeitumgebung für die FastAPI-Anwendung. Der Server startet die API, stellt sie über einen definierten HTTP-Port zur Verfügung und verwaltet eingehende Anfragen, indem er diese an FastAPI weiterleitet und die Antworten zurückgibt. Durch die hohe Performance von Uvicorn kann das Backend auch bei mehreren gleichzeitigen Anfragen effizient arbeiten, ohne dass es zu merklichen Verzögerungen kommt.

3.3.4 Pydantic

Für die Datenvalidierung und strukturierte Modellierung von Datenstrukturen wird die Python-Bibliothek Pydantic ausgewählt. Pydantic ermöglicht die Definition von Datenmodellen, die beim Empfang oder Versand von Daten automatisch validiert und bei Bedarf konvertiert werden. Im Health-Monitoring-System kommen diese Modelle vor allem bei der Verarbeitung von API-Requests und Responses zum Einsatz, wo sie sicherstellen, dass alle übermittelten Daten dem erwarteten Format entsprechen. [9]

Automatische Datenvalidierung

Ein zentrales Merkmal von Pydantic ist die automatische Validierung eingehender Daten. Sobald ein Client eine Anfrage an die API sendet, prüft Pydantic, ob alle erforderlichen Felder vorhanden sind und ob diese den definierten Datentypen entsprechen. Wird etwa erwartet, dass ein bestimmtes Feld eine Zeichenkette enthält oder ein Datum in einem

vorgegebenen Format übergeben wird, erkennt Pydantic Abweichungen davon sofort und gibt eine entsprechende Fehlermeldung zurück. Fehlerhafte Eingaben werden damit bereits an der Systemgrenze abgefangen, bevor sie in die eigentliche Verarbeitungslogik gelangen. [10]

Datenmodelle im System

Pydantic verwendet sogenannte Model-Klassen, um die Struktur von JSON-Objekten zu beschreiben, die über die API übertragen werden. Im Health-Monitoring-System werden auf diese Weise Modelle für Health-Check-Ergebnisse, Kundeninformationen, Task-Objekte und Ticket-Daten definiert. Diese einheitliche Strukturierung stellt sicher, dass Daten innerhalb des gesamten Systems konsistent repräsentiert werden und keine unerwarteten Formate in der Verarbeitungslogik auftreten.

Integration mit FastAPI

Ein besonderer Vorteil von Pydantic im Kontext dieses Projekts ist die native Integration mit FastAPI. Da FastAPI Pydantic-Modelle direkt für die Definition von Request- und Response-Schemata verwendet, entsteht eine durchgehend typsichere API, bei der Validierung, Dokumentation und Schnittstellendefinition aus einer gemeinsamen Modellbeschreibung abgeleitet werden. Das reduziert redundanten Code und trägt zur Konsistenz der gesamten Backend-Architektur bei.

3.4 Cloud & Datenerhaltung

3.4.1 Google BigQuery

Für die Speicherung und Analyse der Monitoring-Daten wird Google BigQuery eingesetzt - ein von Google verwaltetes Data-Warehouse-System, das speziell für die Verarbeitung großer Datenmengen ausgelegt ist. Da das Gesamtsystem ohnehin auf der Google Cloud Platform betrieben wird, fügt sich BigQuery nahtlos in die bestehende Infrastruktur ein und war daher eine logische Wahl für diesen Einsatzzweck.

Im Health-Monitoring-System dient BigQuery als zentrale Datenquelle für die Statusinformationen aller überwachten Kunden-Apps. Die anfallenden Monitoring-Daten

werden dort regelmäßig aggregiert und können anschließend per SQL-Abfrage vom Backend abgerufen werden. Ein wesentlicher Vorteil dabei ist die serverlose Architektur von BigQuery: Skalierung, Speicherverwaltung und Performanceoptimierung werden vollständig von der Plattform übernommen, sodass kein Aufwand für den Betrieb einer eigenen Datenbankinfrastruktur entsteht. [11]

Architektur und Funktionsweise

Die Eignung von BigQuery für analytische Workloads lässt sich auf die zugrunde liegende Architektur zurückführen. BigQuery verwendet eine sogenannte Columnar Storage Architektur, bei der Daten spaltenweise statt zeilenweise gespeichert werden. [12] Für analytische Abfragen ist das ein entscheidender Vorteil: Da typischerweise nur ein Teil der vorhandenen Spalten benötigt wird, müssen nicht alle gespeicherten Daten gelesen werden, was Abfragen erheblich beschleunigt und gleichzeitig eine effizientere Datenkompression ermöglicht. Darüber hinaus verteilt BigQuery die Verarbeitung von Abfragen automatisch auf viele parallele Rechenknoten, sodass auch komplexe Analysen über sehr große Datensätze in kurzer Zeit abgeschlossen werden können.

Einsatz im Monitoring-System

Im Health-Monitoring-System werden mehrere BigQuery-Tabellen genutzt, die unterschiedliche Aspekte des Systems abdecken: Health-Check-Ergebnisse der Apps, HubSpot-Ticket-Daten, App-Konfigurationsinformationen sowie Monitoring-Daten aus dem App Store und Google Play. Diese Aufteilung in thematisch getrennte Tabellen ermöglicht es, Daten gezielt abzufragen, ohne bei jeder Anfrage unnötige Informationen laden zu müssen.

Der Zugriff auf BigQuery erfolgt im Backend über die offizielle Python-Clientbibliothek. Die SQL-Abfragen werden dabei dynamisch zur Laufzeit generiert und ausgeführt, abhängig davon, welche App-Daten jeweils benötigt werden. Die zurückgelieferten Ergebnisse werden anschließend im Backend aufbereitet und als strukturierte Daten an das Frontend-Dashboard weitergegeben.

Vorteile für das Projekt

BigQuery erweist sich für dieses Projekt aus mehreren Gründen als geeignete Wahl. Die serverlose Architektur reduziert den Betriebsaufwand erheblich, da keine Infrastruktur manuell verwaltet werden muss. Analytische SQL-Abfragen können auch bei wachsenden Datenmengen performant ausgeführt werden, ohne dass Anpassungen am System erforderlich werden. Nicht zuletzt vereinfacht die enge Integration in die Google Cloud Platform die Verwaltung von Authentifizierung und Zugriffsrechten, da diese zentral über die GCP-Infrastruktur abgewickelt werden.

3.4.2 Google Cloud Platform

Die Infrastruktur des Health-Monitoring-Systems basiert auf der Google Cloud Platform (GCP), Googles umfassender Cloud-Computing-Plattform. GCP stellt eine breite Palette an Diensten bereit - von Hosting und Datenspeicherung über Datenverarbeitung bis hin zu KI-Funktionalitäten - und bildet das gemeinsame Fundament, auf dem die verschiedenen Systemkomponenten betrieben und miteinander verbunden werden. [13]

Cloud Computing als Grundlage

Die Entscheidung, das System auf einer Cloud-Plattform aufzubauen, folgt einem in der modernen Softwareentwicklung weit verbreiteten Ansatz: Anstatt eigene Server zu betreiben, werden IT-Ressourcen als Service über das Internet bezogen. [14] Für ein Projekt dieser Art bietet das konkrete Vorteile - insbesondere, weil der Entwicklungsaufwand so auf die eigentliche Anwendungslogik konzentriert werden kann, ohne dass Infrastrukturthemen im Vordergrund stehen. Darüber hinaus ermöglicht die Cloud eine hohe Verfügbarkeit und lässt sich bei steigendem Bedarf ohne zusätzliche Hardwareanschaffungen skalieren.

Integration im System

Im Rahmen des Projekts wird GCP vor allem für drei Zwecke genutzt: den Zugriff auf BigQuery als zentrale Datenquelle, die Authentifizierung der Backend-Komponenten über Service Accounts sowie die Verwaltung von API-Zugriffsrechten zwischen den einzelnen Diensten. Das Konzept der Service Accounts spielt dabei eine zentrale Rolle:

Anstatt Zugangsdaten direkt im Code zu hinterlegen, erhält das Backend über einen dedizierten Service Account genau jene Berechtigungen, die für den Betrieb notwendig sind. Dieses Vorgehen folgt dem Prinzip der minimalen Rechtevergabe und trägt sowohl zur Sicherheit als auch zur Wartbarkeit des Systems bei.

Sicherheit und Skalierbarkeit

Ein weiterer Aspekt, der für den Einsatz von GCP spricht, ist die integrierte Sicherheitsarchitektur der Plattform. Über das IAM-System (Identity and Access Management) können Zugriffsrechte granular und rollenbasiert vergeben werden, sodass jede Systemkomponente ausschließlich auf die Ressourcen zugreifen kann, für die eine entsprechende Berechtigung vorliegt. Ergänzt wird dies durch verschlüsselte Datenübertragung sowie integrierte Monitoring- und Logging-Funktionen, die einen kontinuierlichen Einblick in den Systemzustand ermöglichen. Diese Eigenschaften erleichtern nicht nur den sicheren Betrieb, sondern auch die spätere Erweiterung des Systems, da neue Komponenten ohne großen Aufwand in die bestehende Infrastruktur eingebunden werden können.

3.4.3 SQL

Für den Zugriff auf die in BigQuery gespeicherten Monitoring-Daten wird SQL (Structured Query Language) verwendet. SQL ist die etablierte Standardsprache für relationale Datenbanken und ermöglicht es, Daten gezielt abzufragen, zu filtern, zu aggregieren und miteinander zu verknüpfen. [15, 16] Da BigQuery SQL nativ unterstützt und die Sprache ausdrucksstark genug ist, um die im Monitoring-System benötigten Auswertungen präzise umzusetzen, war ihr Einsatz in diesem Kontext naheliegend.

Grundlagen von SQL

SQL wurde in den 1970er Jahren entwickelt und hat sich seither als universeller Standard für die Arbeit mit strukturierten Daten etabliert. Die Sprache umfasst Befehle für alle grundlegenden Datenbankoperationen - SELECT zum Abfragen, INSERT zum Einfügen, UPDATE zum Ändern und DELETE zum Entfernen von Datensätzen. Im analytischen Umfeld steht dabei der SELECT-Befehl klar im Vordergrund, da dort primär Daten ausgewertet und zusammengefasst werden, ohne den Datenbestand selbst zu verändern.

Einsatz im Projekt

Im Monitoring-System werden SQL-Abfragen hauptsächlich dazu verwendet, Health-Check-Daten und Ticket-Informationen aus BigQuery abzurufen, nach bestimmten Kunden-Apps zu filtern und die Ergebnisse sinnvoll zu gruppieren. Da sich die abgefragten Apps und Zeiträume je nach Anfrage unterscheiden, werden die Abfragen im Backend dynamisch zur Laufzeit generiert und ausgeführt. Die zurückgelieferten Ergebnisse werden anschließend in JSON-Strukturen überführt, die vom Frontend direkt verarbeitet werden können.

Vorteile für die Datenanalyse

SQL eignet sich für diesen Anwendungsfall besonders gut, weil es eine sehr präzise Kontrolle darüber ermöglicht, welche Daten in welcher Form abgerufen werden. Komplexe Filterungen, Aggregationen und Verknüpfungen lassen sich direkt in der Abfrage ausdrücken, ohne dass dafür umfangreiche Verarbeitungslogik im Backend implementiert werden müsste. In Kombination mit BigQuery können auf diese Weise auch größere Datensätze effizient ausgewertet werden - die rechenintensive Verarbeitung übernimmt BigQuery, während sich das Backend auf die Weiterverarbeitung und strukturierte Aufbereitung der Ergebnisse konzentriert.

3.5 Externe Schnittstellen

3.5.1 Asana API

Zur Integration von Aufgabenmanagement-Daten wird die API der Projektmanagementplattform Asana verwendet. Über diese Schnittstelle können Tasks programmatisch erstellt, abgefragt und aktualisiert werden, ohne dass ein manueller Eingriff in die Plattform selbst erforderlich ist. [17]

Im Health-Monitoring-System wird die Asana API konkret dazu genutzt, offene Aufgaben zu bestimmten Kunden-Apps abzurufen und im Dashboard darzustellen. Das ermöglicht es, auf einen Blick zu erkennen, ob zu einem erkannten Problem bereits Maßnahmen eingeleitet wurden - eine Information, die für die Priorisierung von Reaktionen im Support-Prozess relevant ist.

API-Funktionalität

Die Asana API bietet Zugriff auf die zentralen Elemente des Projektmanagementsystems, darunter Projekte, Tasks, Kommentare, Benutzer und benutzerdefinierte Felder. Für das Monitoring-System sind dabei vor allem Tasks und deren Status von Bedeutung, da diese den aktuellen Bearbeitungsstand zu einem Problem widerspiegeln.

Integration im Monitoring-System

Die Kommunikation mit der Asana API erfolgt im Backend über HTTP-Anfragen. Dabei werden sowohl bestehende Aufgaben abgerufen als auch bei Bedarf neue erstellt. Jede Aufgabe enthält einen Titel, eine Beschreibung, eine Zuordnung zu einem Kunden-Slug sowie einen Verantwortlichen. Diese Informationen werden vom Backend verarbeitet und anschließend strukturiert im Monitoring-Dashboard dargestellt, sodass der Zusammenhang zwischen einer App und den zugehörigen offenen Aufgaben direkt ersichtlich ist.

3.5.2 HubSpot API

Neben den Aufgabendaten aus Asana werden über die HubSpot API auch Support- und CRM-Daten in das Monitoring-System integriert. HubSpot wird im Unternehmen zur Verwaltung von Kundenanfragen und Support-Tickets eingesetzt, und diese Informationen sind für ein vollständiges Bild des App-Status ebenso relevant wie technische Monitoring-Daten. [18]

CRM-Integration

CRM-Systeme (Customer Relationship Management) dienen der zentralen Verwaltung von Kundenbeziehungen und Supportanfragen. Durch die Einbindung dieser Daten in das Monitoring-System entsteht ein umfassenderer Überblick über den Zustand einer Kunden-App: Technische Auffälligkeiten lassen sich so direkt mit bereits bekannten Supportfällen in Verbindung bringen, was die Einschätzung der Lage erheblich erleichtert.

Verwendung im Dashboard

Konkret wird die HubSpot API dazu verwendet, offene Support-Tickets zu zählen und bestimmten Kunden zuzuordnen. Der daraus resultierende Supportstatus wird im Dashboard visualisiert und gibt dem Nutzer unmittelbar Auskunft darüber, ob zu einer App bereits aktive Supportvorgänge existieren. Das verhindert Doppelarbeit und schafft eine gemeinsame Informationsbasis für technische und supportverantwortliche Teams.

3.5.3 Google Play Developer API

Zur Überwachung des Status von Android-Anwendungen wird die Google Play Developer API eingebunden. Diese Schnittstelle ermöglicht den programmatischen Zugriff auf Informationen aus dem Google Play Entwicklerkonto und bildet damit die Grundlage für die automatisierte Überprüfung von Android-Apps im Monitoring-System. [19]

Funktionen der API

Die Google Play Developer API stellt unter anderem Informationen zum Veröffentlichungsstatus einer App, zu aktiven App-Versionen sowie zu etwaigen Richtlinienverstößen bereit. Gerade letztere sind für ein Monitoring-System von besonderer Bedeutung, da Richtlinienverstöße im schlimmsten Fall zur Delistung einer App führen können und daher frühzeitig erkannt werden sollten.

Bedeutung für das Monitoring

Im Health-Monitoring-System wird diese API genutzt, um den aktuellen Status der überwachten Android-Apps kontinuierlich zu prüfen. Probleme wie fehlende Verifizierungen, Einschränkungen der App-Verfügbarkeit oder aktive Richtlinienverstöße werden dabei automatisch erkannt und im Dashboard sichtbar gemacht, sodass zeitnah reagiert werden kann.

3.5.4 App Store Connect API

Analog zur Google Play Developer API wird für iOS-Anwendungen die App Store Connect API verwendet. Diese Schnittstelle bietet Entwicklern programmatischen

Zugriff auf Informationen aus dem Apple Developer Account und ermöglicht damit eine automatisierte Überwachung des Status von iOS-Apps. [20]

API-Funktionalität

Über die App Store Connect API lassen sich App-Statusinformationen, der Verfügbarkeitsstatus im App Store, App-Metadaten sowie Account-Konfigurationen abrufen. Diese Daten geben Aufschluss darüber, ob eine iOS-App korrekt konfiguriert ist und ohne Einschränkungen im App Store verfügbar ist.

Einsatz im Monitoring-System

Im Monitoring-System wird diese API genutzt, um potenzielle Probleme mit iOS-Apps frühzeitig zu identifizieren. Dazu zählen etwa fehlende App-Store-Verfügbarkeit, Konfigurationsprobleme im Developer Account oder Einschränkungen durch Apple-Richtlinien. Da diese Prüfungen automatisiert im Hintergrund ablaufen, werden entsprechende Auffälligkeiten unmittelbar im Dashboard visualisiert, ohne dass eine manuelle Kontrolle des Developer Accounts erforderlich ist.

4 Umsetzung

4.1 UI und UX

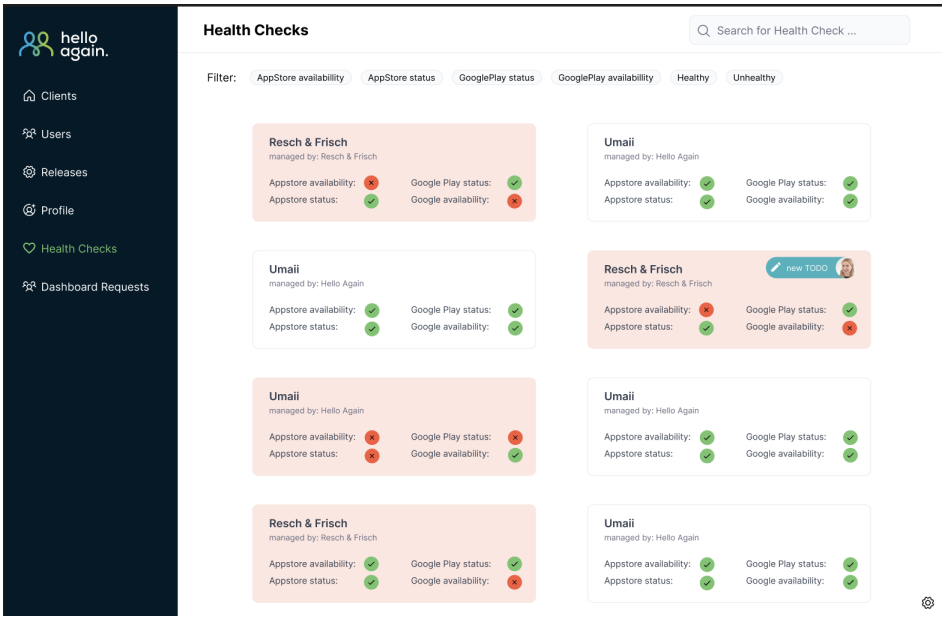
4.1.1 UI und UX Design

Das Design des Frontends wurde zu Beginn in Figma ausgearbeitet und weiters durch absprache und Feedback-Runden verbessert.

Iteration 1:

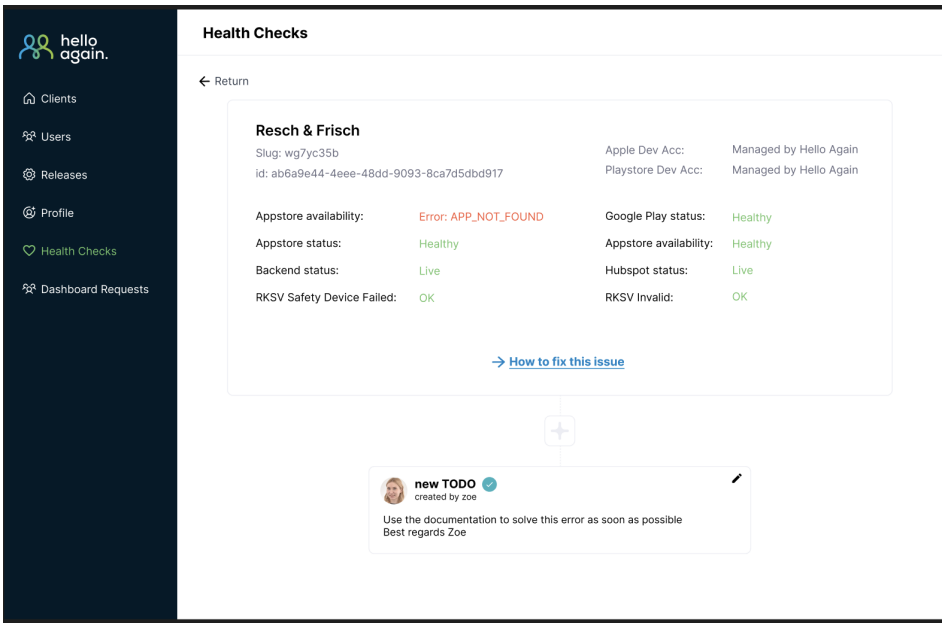
Im ersten Meeting mit der Firma wurden uns die Anforderungen für unsere Anwendung dargelegt und es wurde gebeten, zwei verschiedene Workflows zu entwickeln. Dabei benötigen wir eine Übersichtsseite und eine Detailseite sowie einen Weg um zur Dokumentation in Confluence zu kommen und weiters eventuell auch Statistiken.

Workflow-Version 1



Dies ist die erste Version der Übersichtsseite. Hier werden die verschiedenen Kunden angezeigt, sowie ein grober Überblick über die Health-Stati. Je nach Farbe wird angezeigt

ob die jeweiligen Apps Healthy oder Unhealthy sind (also ob sie Error werfen oder nicht). Sobald eine App ausfällt wird sie sofort gekennzeichnet, damit die Mitarbeiter auch dementsprechend auf diese Ausfälle reagieren können.



Dies ist die erste Version der Detailseite, auf der nochmal genauere Daten der Kunden angezeigt werden, sowie den Health-Status der einzelnen Health-Checks. Wird ein Fehler gemeldet wird auf der Detailsseite auch automatisch der Link angezeigt zur Lösung des Errors. Dies ermöglicht den Mitarbeitern eine effiziente und schnelle Arbeitsweise. Weiter unsen sehen wir noch eine TODO-Funktion. Hier kann man direkt im Dashboard Kommentare schreiben, die dann Plattformübergreifend in Asana erstellt werden.

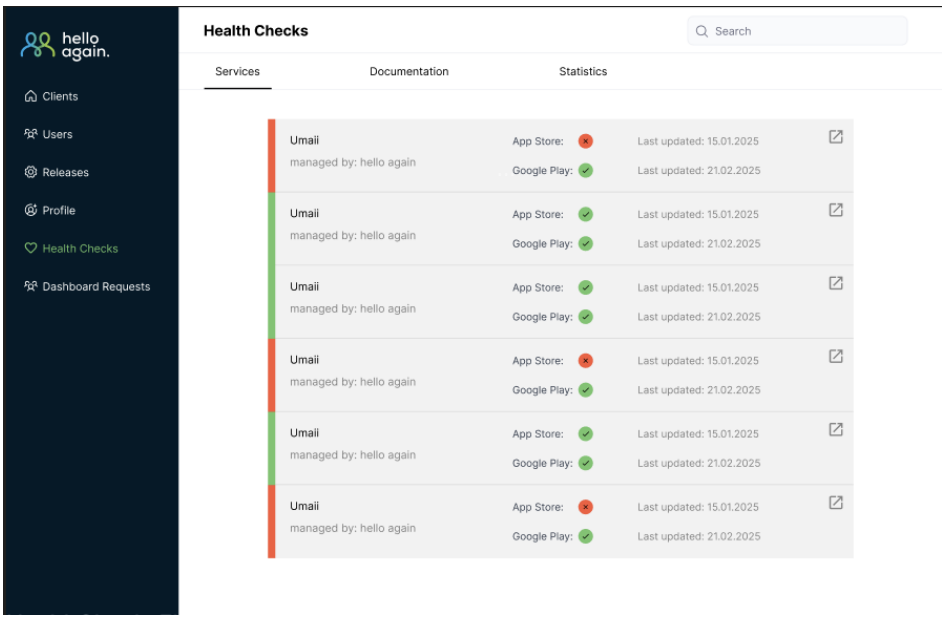
Workflow-Version 2

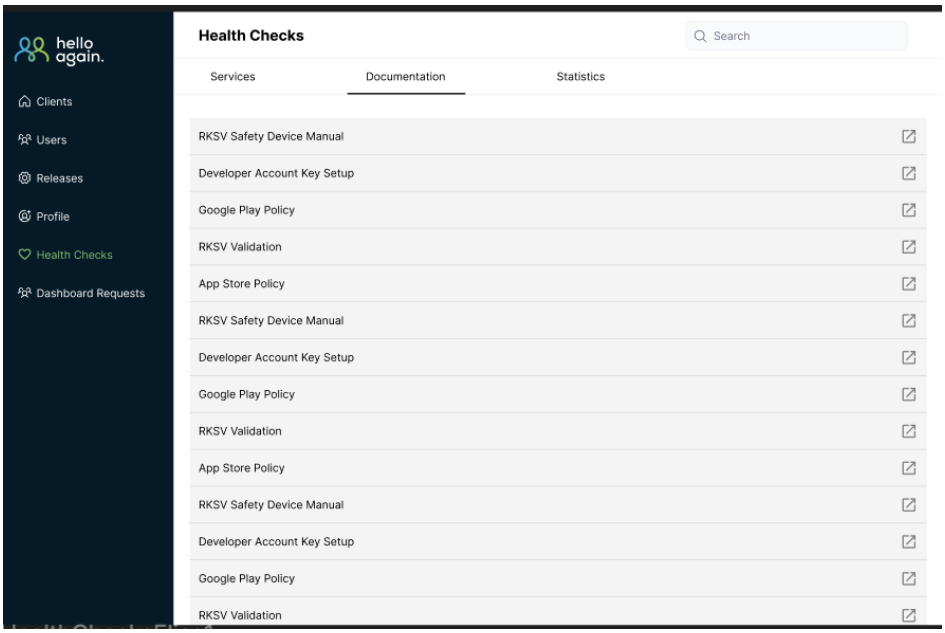
Dies ist die zweite Workflow-Version die erarbeitet wurde. Hier haben wir ebenfalls eine Übersichtsseite mit einer Liste doch mit einem kleinen Unterschied.

Um zu Dokumentationsseite zu gelangen kann der Tab gewechselt werden, allerdings werden hier alle vorhandenen Dokumentationen zur Fehlerbehebung angezeigt.

Als dritten Tab wurde eine Statistik-Übersicht eingebaut bei der man die Downloads der letzten Tage sieht, sowie die Anzahl der App-Ausfälle, damit man potenzielle Fehlerquellen zeitnahe erkennen und beseitigen kann.

Die Detailseite sieht sehr ähnlich aus zum ersten Workflow-Konzept allerdings mit dem Unterschied, dass man die Detailseite als Popup sieht. Unten rechts erscheint ein Button

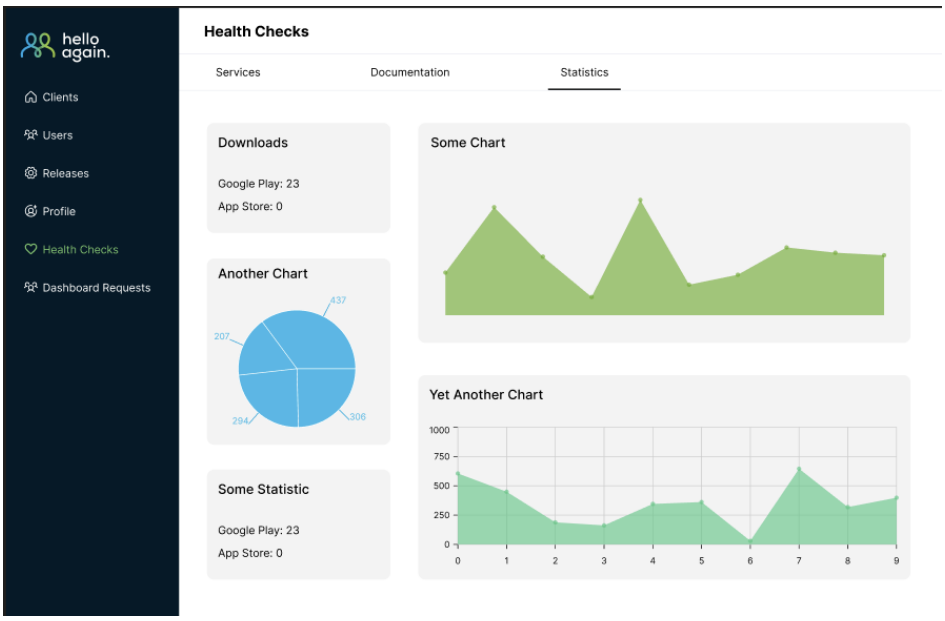


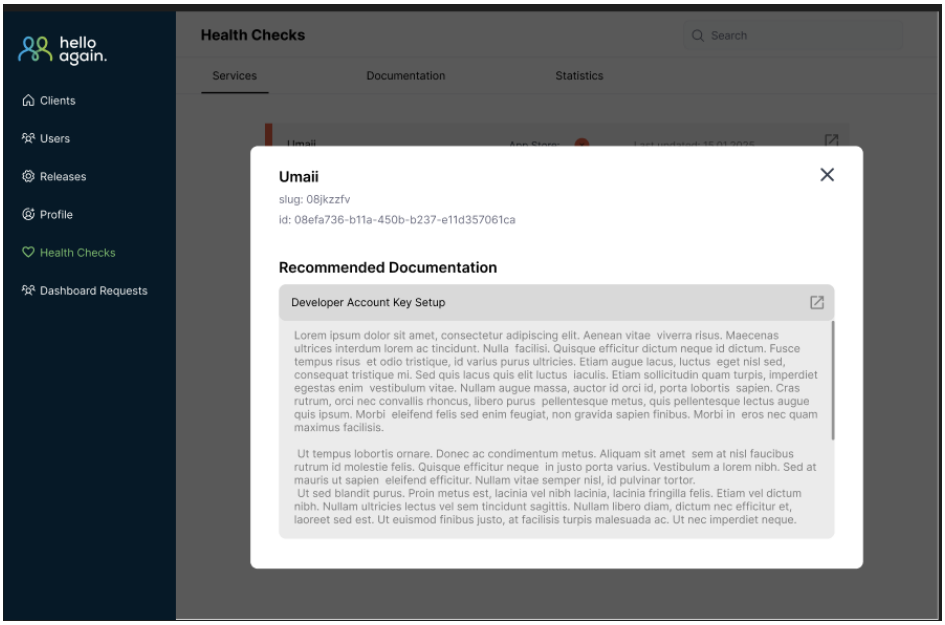


mit dem Namen Resolve Errors. Klickt man auf diesen, wird ein Fenster angezeigt, dass uns die Dokumentation zur Fehlerbehebung liefert.

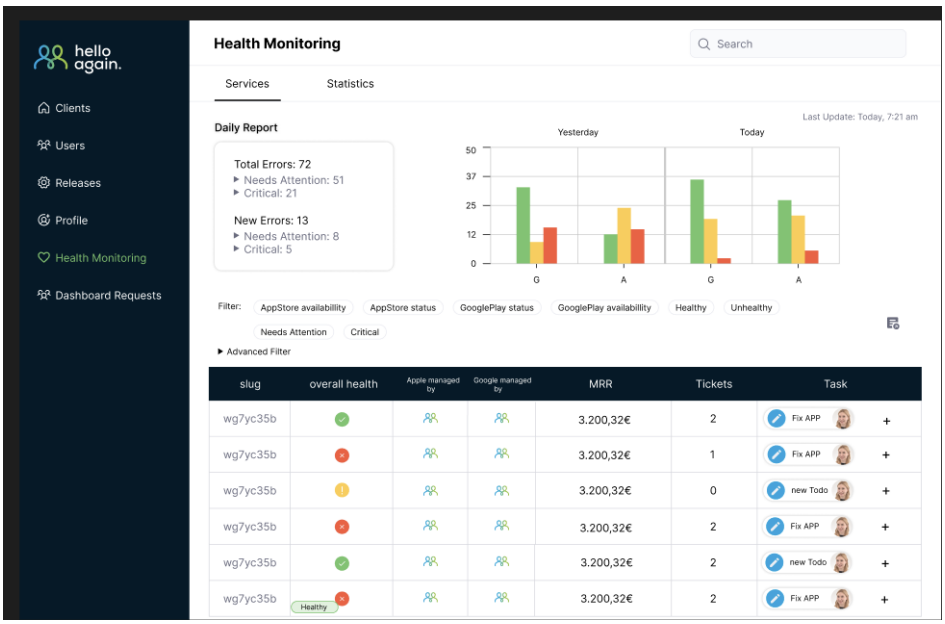
Iteration 2:

Zur Vorstellung beider Konzepte wurden wir in die Firma eingeladen und haben diese dort auch Vorgestellt. Es wurde schnell klar, dass wir eine Übersichtsseite sowie eine Detailseite brauchen würden mit einer erweiterten Kommentarfunktion und einem Link zur externen Fehlerbehebung, darum wurde sich auf eine Misch-Version der beiden Workflows geeinigt. Allerdings musste die Liste aller Kunden auf der Übersichtsseite noch etwas schmaler und informativer werden.

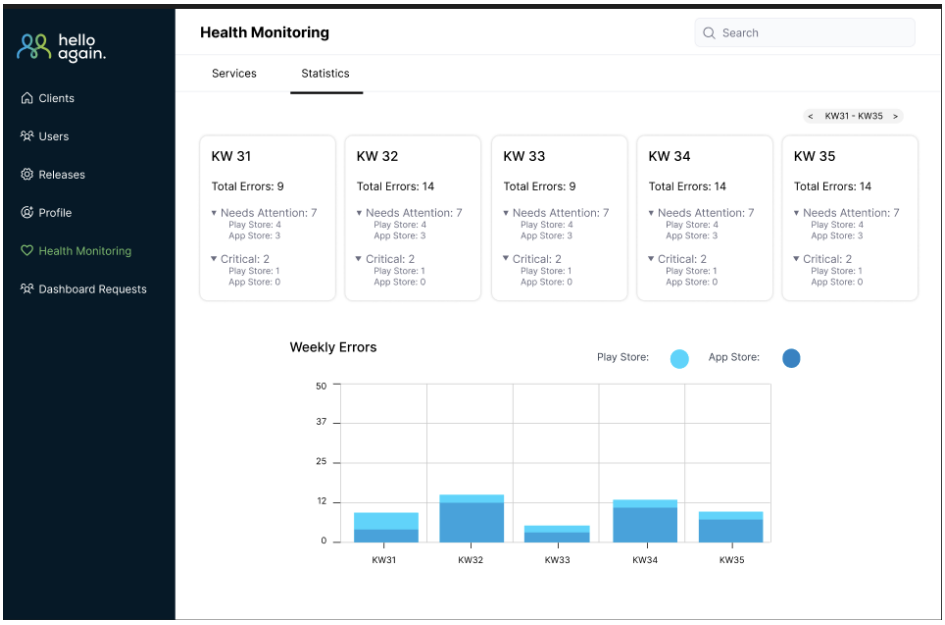




Hier sehen wir das verbesserte Dashboard-Design, dass nach dem ersten Feedback-Gespräch zustande kam. Es wurde eine Übersichtliche Statistik eingearbeitet sowie eine schmalere Liste mit mehr nützlichen Informationen für die Mitarbeiter. Dazu wurden noch Quickfilter eingebaut, die ermöglichen sollen, dass man schnell alle fehlerhaften Apps herausfiltern kann, damit die Zeit, die die App nicht im Appstore verfügbar ist, so klein wie möglich gehalten wird, um so wenig potenziellen Umsatz wie möglich zu verlieren. Dazu gibt es noch die Advanced-Filter Option. Diese ist dafür da um nach konkreten Fehlern filtern zu können. Wenn man zum Beispiel weiß, dass das Liscenseagreement von Google geändert wurde, kann man mit den Advanced-Filtern danach suchen und diese Fehler beheben. Beim Overall-Health-Status wurde noch eine dritte Option eingführt neben Healthy und Healthy. Diese heißt Needs-Attention, dass

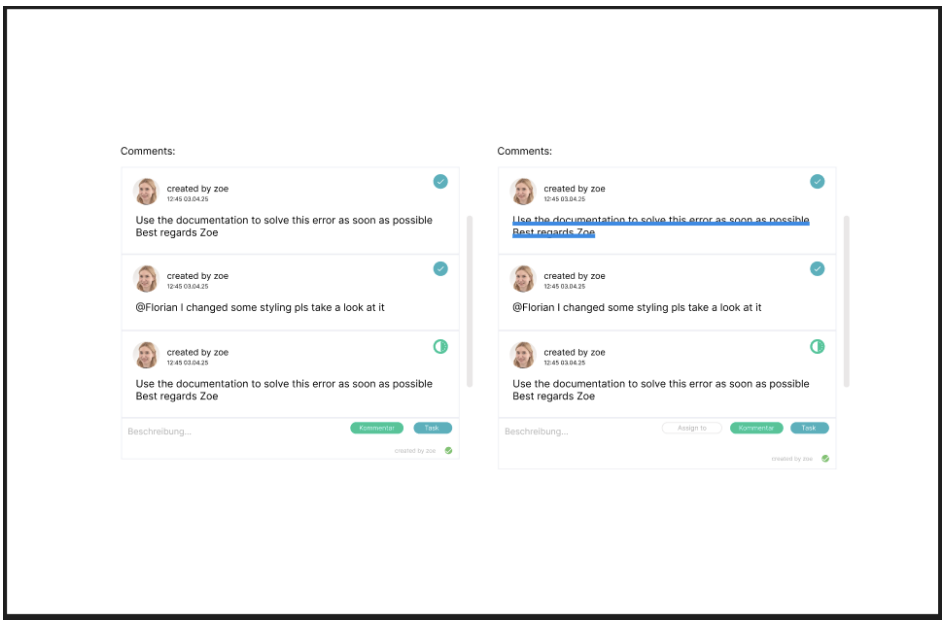


heißt die App könnte bald aus dem Appstore oder Googleplaystore entfernt werden, da Lizenzen abgelaufen sind. Auch ein Schnellzugriff auf die TODOS soll gewährleistet sein.



Weiters gibt es einen eigenen Tab, der die Statistiken auf der Übersichtsseite nochmals erweitert und mit mehr Informationen darstellt.

Dazu wurde die Kommentar- und TODO-Funktion nochmals überarbeitet in zwei verschiednen Version, hier wurde sich schlussendlich für diese Version (siehe Bild oben) entschieden.



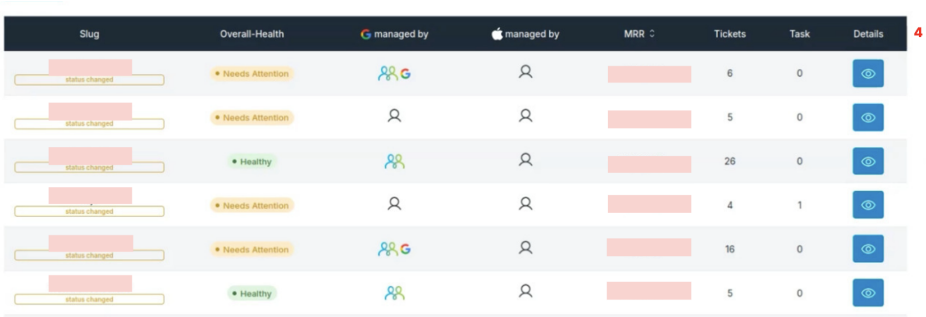
4.2 Dashboard

4.2.1 Frontend

Übersichtsseite

Tabellenansicht

Zu Beginn wurde die Tabelle erstellt auf der Übersichtsseite, hier werden die wichtigsten Daten der Kunden, für die Mitarbeiter dargestellt. Man muss beachten, dass das Design der finalen Version des Dashboards von der Figma Version abweicht, da die Firma intern Code guidelines hat, an die wir uns halten mussten. Ebenfalls gab es styling Guidelines und Komponenten die uns zu Verfügung gestellt wurden, wie zum Beispiel die Pagination.



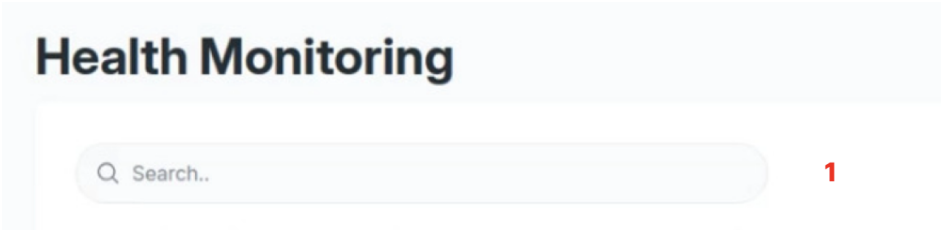
Hier werden angezeigt:

- der Slug (Eindeutiger Kundenbezeichner; zeigt auch ein status changed-Tag, falls sich der Status geändert hat)

- Overall-Health (Ampelstatus des Clients (Healthy, Needs Attention, Unhealthy, Critical))
- Google managed by (Zuständiges Team/Person für den Google-Account)
- Apple managed by (Zuständiges Team/Person für den Apple-Kanal)
- MRR (Monthly Recurring Revenue)
- Tickets (Anzahl offener Support-Tickets)
- Task (Anzahl offener Aufgaben)
- Details (Link zur Detailseite)

Da die Tabelle das Herzstück des Dashboards ist wurde sie als erstes implementiert und alles andere darauf aufgebaut. Bei der Tabelle wurde auch daraus geachtet, dass die Columns filterbar sind nach dem höchsten oder niedrigsten MRR.

Suchfunktion



Als nächstes wurde eine Suchfunktion implementiert, bei der man nach dem Slug filtern kann. Die Suchfunktion ist hierbei caseinsensitive.

Quickfilter

Um den Mitarbeitern eine möglichst schnelle Filteroption zu bieten, gibt es die Quickfilter, diese sind als multiselect implementiert worden. Das heißt man kann zum Beispiel nach allen Apps filtern, die unhealthy sind und mit status changed geflagged sind. Das bedeutet, die App war gestern noch auf Healthy oder Needs Attention und ist heute aus den Stores geflogen.

Was bedeuten die Quickfilter Optionen?

- Healthy (Die App weist keine Fehler auf)
- Needs Attention (Die App wird bald aus den Stores entfernt)

- Critical (Die App wurde aus den Stores entfernt und ist nichtmehr verfügbar)
- Unhealthy (Der Status der App ist entweder Needs Attention oder Critical. Hier muss etwas getan werden!)
- Google Availability Issues (Der Google Playstore weist Probleme bei der App auf)
- Apple Availability Issues (Der Appstore weist Probleme bei der App auf)
- Status Changed (Der Status hat sich zum Vortag verändert)

Das Filtern wird hier vom Frontend übernommen, da wir alle Daten der Tabelle benötigen und diese schon in das Frontend geladen haben. Würden wir wieder eine Request an das Backend schicken, würden wir unnötig Zeit des Benutzers verschwenden.

Advanced Filter

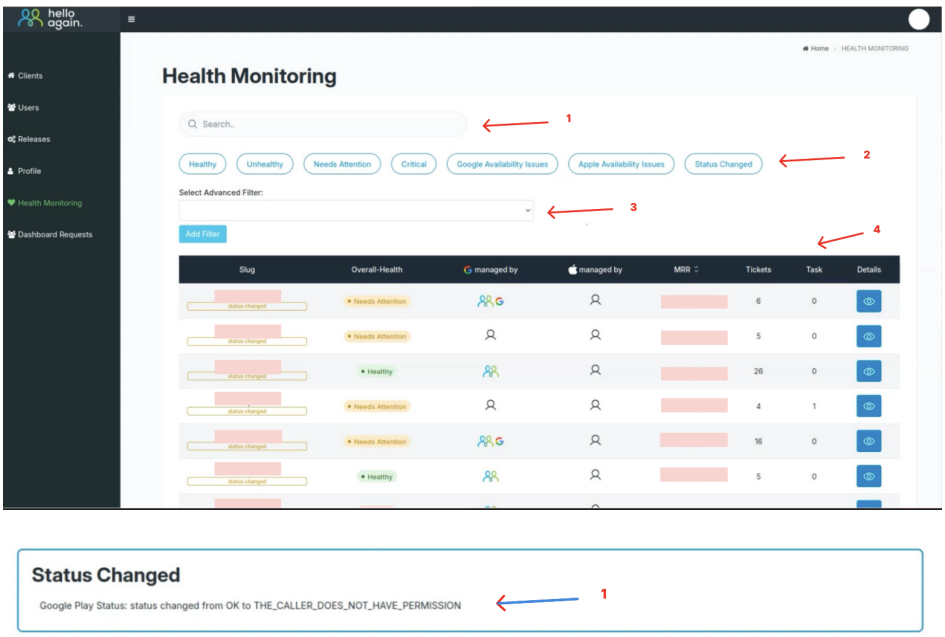
Die Advanced Filter wurden erst zum Schluss implementiert, da diese Filteroption durch das Backend passiert. Wir bekommen alle vorhandenen Fehler die vorkommen vom Backend ins Frontend und diese werden als zweiteilige Filteroption dargestellt. Zuerst wählt man aus nach was man Filtern möchte zum Beispiel Google Availability und danach muss man auswählen welchen spezifischen Error Code man ansehen möchte. Hier auch wieder ein Beispiel man hat Google Availability ausgewählt mit dem spezifischen Error Code License Agreement ran out. Das Frontend hat keinen Zugriff auf die Error Codes der einzelenen Apps, daher wird eine Anfrage an das Backend geschickt mit dem Error Code nachdem gefiltert werden soll. Anschließend wird die Liste neu gefiltert. Da wir eine sehr große menge an Daten durchgehen müssen, dauert dieser Vorgang auch dementsprechend lange.

Finale Version der Übersichtsseite

Detailseite

Status Changed

Am Anfang der Detailseite befindet sich die Status-Changed Flag. Dieser wird nochmal visuell hervorgehoben und zeigt an wie sich die App im gegensatz zum Vortag verändert



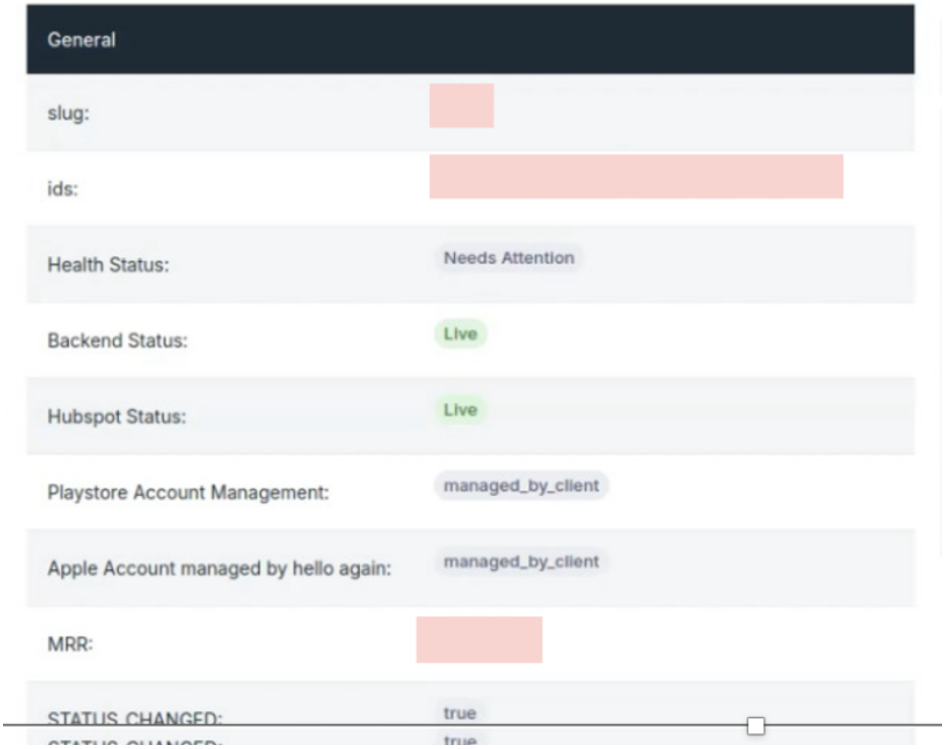
hat. Sollte die App den Status von Healthy zu Unhealthy verändern wird der Error Code, der für diesen Fehler verantwortlich ist auch angezeigt. Somit weiß der Benutzer sofort wo das Problem liegt. Status Changed wird allerdings auch angezeigt, wenn die App sich von Unhealthy zu Healthy ändert, so können die Mitarbeiter zum Beispiel nach der App, die gestern noch Fehlerhaft war suchen und sehen auf den ersten Blick, ob sich ihr Status verändert hat. Die Status changed Flags werden hier über eine Fetch-Request ins Frontend geladen.

General Data

Unter dem Status Changed Flag werden nochmals generelle Daten der App angezeigt, die auf der Übersichtsseite schon einsehbar sind. Hier wurde darauf geachtet, dass die Daten jederzeit erweiterbar sind. Das heißt, wenn wir aus dem Backend ergänzend noch YRR Daten bekommen würden, würde das Frontend sie ohne fremdeinwirkung eines Entwicklers darstellen.

Kommentar Sektion

Die Kommentarsektion ist eine der wichtigsten Funktionen der Detailsseite. Hier können Kommentare und Tasks erstellt werden, die ebenso in Asana erstellt werden. Ebenso werden die Kommentare und Tasks, die bereits in Asana vorhanden sind dargstellt. Wenn man einen Task erstellt kann man auch noch einen Assignee wählen, dem dieser Task dann zugeteilt wird. Hierzu bekommen wir aus dem Backend alle Namen der



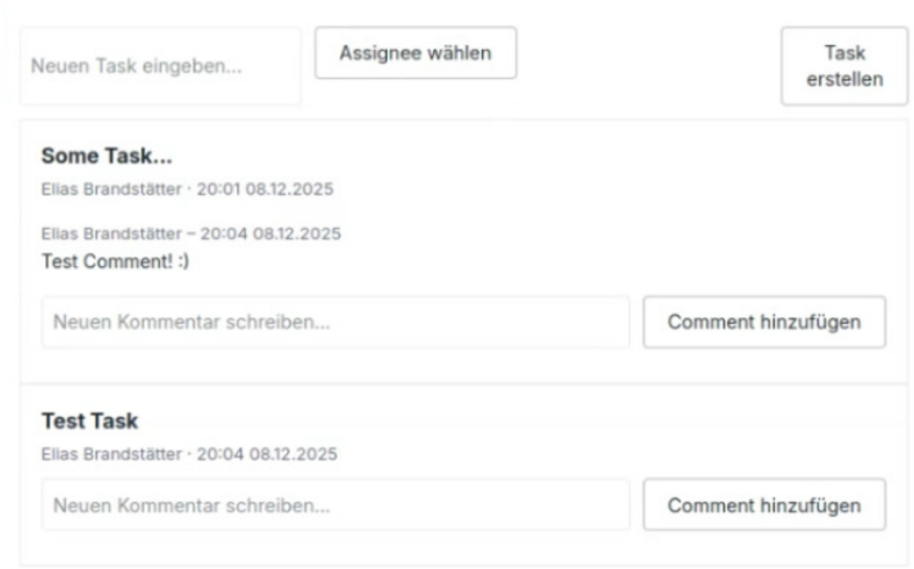
Mitarbeiter, die auf dieses Asana Board Zugriff haben. Wenn wir einen Task erstellen, wird der Text, der Slug und der Name des Assignees an das Backend übergeben, dass dan anschließend den Task in Asana erstellt.

Health-Checks

Eine weitere wichtige Funktion der Detailseite sind die Health-Checks. Hier wird der Status farblich gekennzeichnet, dass man sofort erkennt welche App Probleme wirft. Wenn ein Error vorliegt wird immer dessen Fehlermeldung angezeigt, sowie ein Link zu Fehlerbehebung. Drückt man auf den link wird man zur passenden Fehlerbeheungsdokumentation oder Anleitung in Confluence weitergeleitet. Diese Health-Checks sowie die passenden Links der Doku werden vom Backend bereitgestellt.

Tasks und Tickets

Es wurde außerdem gewünscht, dass die Tasks und Tickets dargestellt werden, da alle 3 miteinander intern verlinkt sind. Daher wird der Text beziehungsweise die Beschreibung der jeweils offenen Tickets angezeigt und die Links zur jeweiligen Plattform. Wichtig ist zu betonen, dass hier nur offenene Tickets dargestellt werden.



Finale Version der Detailsseite

Dies ist die finale Endversion der Detailseite.

4.3 Systemarchitektur

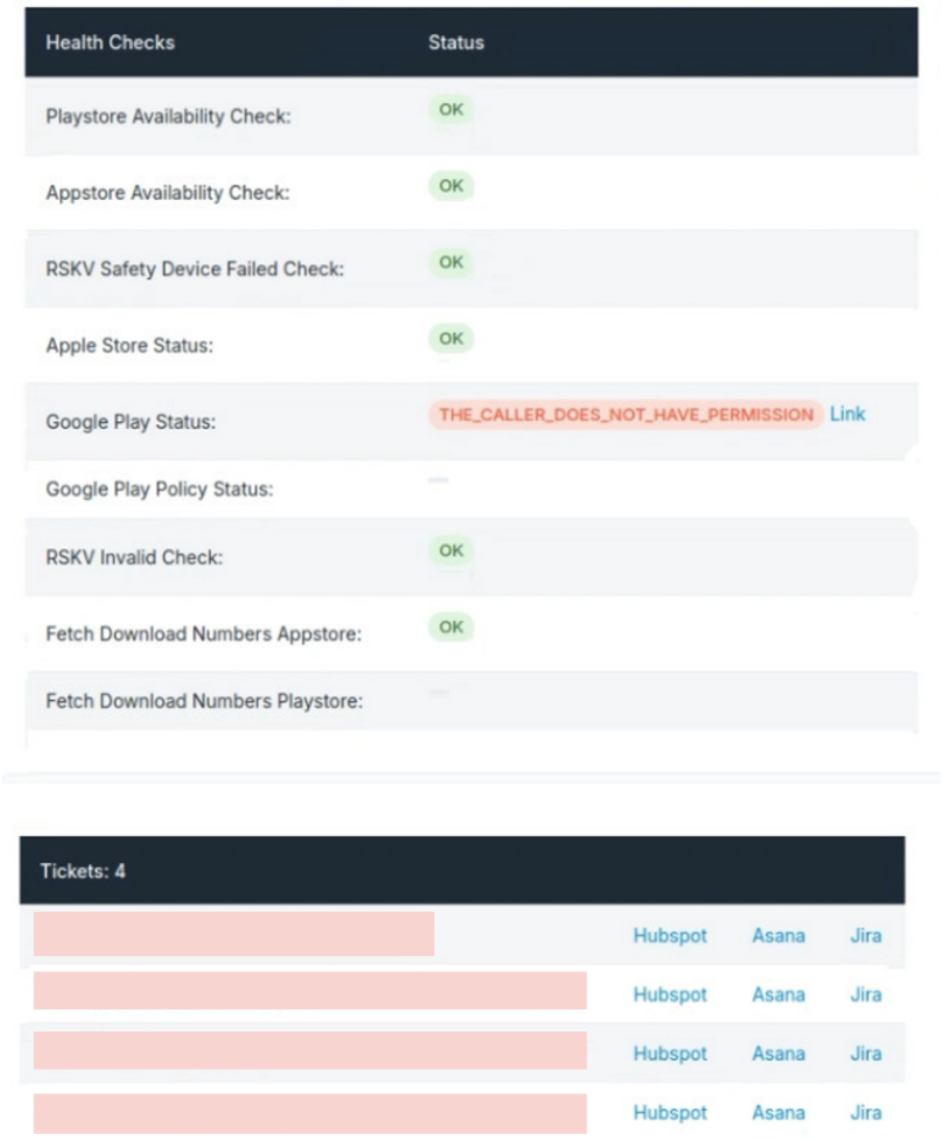
Die Systemarchitektur des Health-Monitoring-Systems folgt einer mehrschichtigen Architektur, die aus einem Frontend, einem Backend sowie mehreren externen Datenquellen besteht. Ziel dieser Architektur ist es, Daten aus verschiedenen Plattformen zentral zu sammeln, zu analysieren und anschließend in einem übersichtlichen Dashboard darzustellen.

Durch die klare Trennung der einzelnen Komponenten wird eine modulare und wartbare Systemstruktur erreicht. Jede Schicht des Systems erfüllt eine spezifische Aufgabe und kommuniziert über definierte Schnittstellen mit den anderen Komponenten.

4.3.1 Gesamtübersicht der Architektur

Das System besteht aus vier zentralen Ebenen:

- Frontend (Dashboard)
- Backend (API und Datenverarbeitung)
- Cloud- und Datenebene
- Externe Schnittstellen

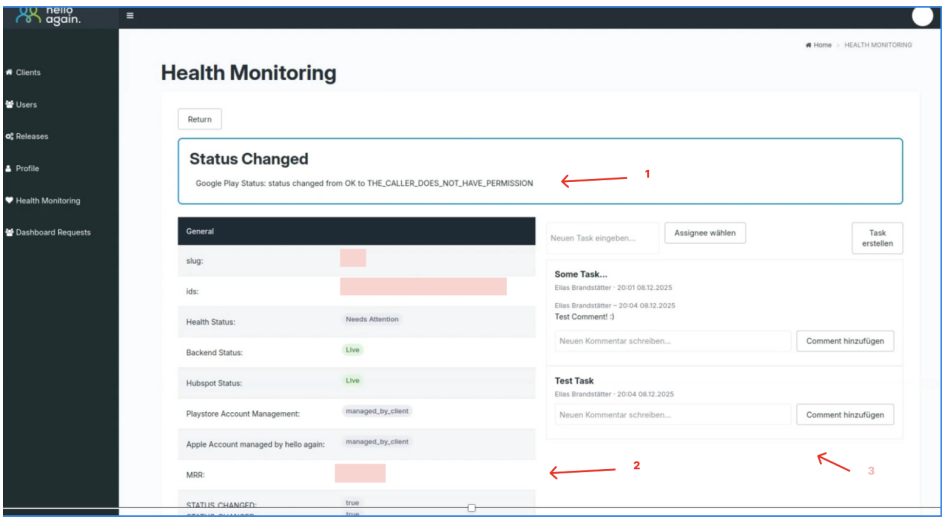


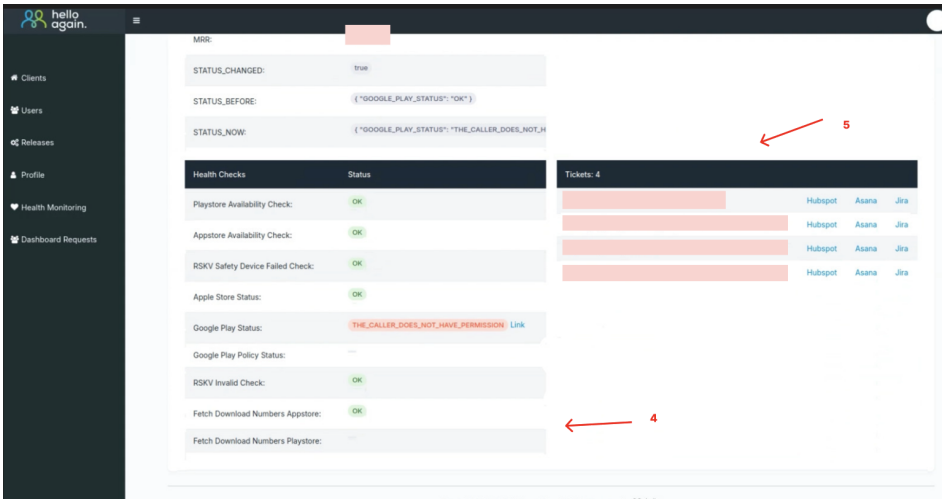
Das Frontend dient als Benutzeroberfläche für Mitarbeiter des Unternehmens und ermöglicht die Anzeige sowie Analyse des Status aller Kunden-Apps. Das Backend stellt eine REST-API bereit, über die das Frontend Daten abrufen kann. Gleichzeitig übernimmt das Backend die Verarbeitung und Aggregation der Daten aus verschiedenen Quellen.

Die Cloud- und Datenebene stellt die zentrale Datenbasis dar, während externe Schnittstellen zusätzliche Informationen aus Drittplattformen liefern.

4.3.2 Frontend-Schicht

Das Frontend stellt die Benutzeroberfläche für die Mitarbeiter dar. Hier wird die Health-Übersicht und die Detailseite der einzelenen Apps dargestellt.





Das Frontend besteht aus:

- UI & UX
- Health-Übersicht
- Detailseite
- Kommentar-Sektion
- Erkennung von Statusänderungen

4.3.3 Backend-Schicht

Das Backend stellt die zentrale Logik des Systems dar und fungiert als Vermittler zwischen Frontend, Datenbank und externen Schnittstellen. Die Backend-Komponente wurde mit FastAPI entwickelt und läuft auf dem ASGI-Server Uvicorn.

Die wichtigsten Aufgaben des Backends sind:

- Bereitstellung einer REST-API für das Frontend

- Abrufen und Verarbeiten von Daten aus Google BigQuery
- Integration externer APIs
- Durchführung von Statusanalysen
- Erkennung von Statusänderungen

Das Backend aggregiert Daten aus mehreren Quellen und stellt sie in strukturierter Form bereit. Diese Daten werden anschließend vom Frontend dargestellt.

Ein wichtiger Bestandteil der Backend-Logik ist die Analyse der Statuswerte der einzelnen Apps. Hierbei werden verschiedene Health-Checks durchgeführt und deren Ergebnisse miteinander kombiniert, um einen Gesamtstatus zu bestimmen.

4.3.4 Daten- und Cloud-Schicht

Die zentrale Datenbasis des Systems befindet sich in der Cloud-Infrastruktur der Google Cloud Platform.

Die Monitoring-Daten werden in Google BigQuery gespeichert und über SQL-Abfragen vom Backend abgerufen. BigQuery ermöglicht eine effiziente Verarbeitung großer Datenmengen und eignet sich besonders für analytische Abfragen.

Zusätzlich werden Snapshots der Statusdaten lokal gespeichert, um Veränderungen im Status der Apps über einen bestimmten Zeitraum hinweg nachvollziehen zu können. Dieses Snapshot-System ermöglicht es, Statusänderungen zwischen zwei Zeitpunkten automatisch zu erkennen.

4.3.5 Integration externer Systeme

Ein vitaler Bestandteil der Systemarchitektur ist die Integration externer Plattformen. Das Monitoring-System greift auf mehrere APIs zu, um Informationen über App-Status, Support-Tickets und Aufgaben zu erhalten.

Die wichtigsten integrierten Systeme sind:

- Asana: zur Verwaltung von Aufgaben
- HubSpot: zur Bereitstellung von Support-Tickets
- Google Play Developer API: zur Überprüfung von Android-App-Status
- App Store Connect API: zur Überprüfung von iOS-App-Status

4.3.6 Datenfluss im System

Der typische Datenfluss innerhalb des Systems lässt sich in mehreren Schritten beschreiben:

1. Das Frontend sendet eine Anfrage an das Backend, beispielsweise zur Anzeige aller Health-Checks.
2. Das Backend führt SQL-Abfragen auf BigQuery aus und ruft aktuelle Monitoring-Daten ab.
3. Zusätzlich werden Informationen aus externen APIs wie Asana oder HubSpot abgefragt.
4. Die Daten werden im Backend aggregiert und analysiert.
5. Das Backend sendet die aufbereiteten Daten als JSON-Antwort an das Frontend zurück.
6. Frontend visualisiert die Ergebnisse im Dashboard.

Durch diese Architektur wird sichergestellt, dass alle Daten zentral verarbeitet und konsistent dargestellt werden.

4.3.7 Vorteile der gewählten Architektur

Die gewählte Systemarchitektur bietet mehrere Vorteile:

- Modularität: einzelne Komponenten können unabhängig weiterentwickelt werden
- Skalierbarkeit: Cloud-Infrastruktur ermöglicht Erweiterung bei steigender Datenmenge
- Wartbarkeit: klare Trennung zwischen Frontend, Backend und Datenebene
- Flexibilität: neue APIs oder Datenquellen können einfach integriert werden

Diese Architektur bildet somit eine stabile Grundlage für den Betrieb und die zukünftige Erweiterung des Health-Monitoring-Systems.

4.4 Backend-Implementierung

4.5 Integration externer Systeme

4.6 Datenmanagement

4.7 Fehlerbehandlung und Logging

4.8 Sicherheit und Zugriff

4.8.1 Einleitung

Im Rahmen unserer Diplomarbeit bei Hello Again wurde dem Thema Sicherheit und Datenschutz von Beginn an ein hoher Stellenwert gegeben. Bereits zu Beginn der Zusammenarbeit fand eine Einschulung statt, in der uns die Richtlinien zum sicheren Umgang mit Daten, Systemen und Tools erklärt wurden. Diese Richtlinien betrafen sowohl den Umgang mit sensiblen Kundendaten als auch die erlaubten Tools.

4.8.2 Datenschutz und der Umgang mit Kundendaten

Einer der wichtigsten Punkte betraf den Umgang mit Kundendaten. Uns wurde explizit erklärt, dass Kundendaten zu keinem Zeitpunkt in externe KI-Systeme eingegeben werden dürfen. Konkret wurde dabei ChatGPT als Beispiel genannt; ein öffentlich zugängliches KI-Tool, das von OpenAI betrieben wird. Dieser Grundsatz ist aus mehreren Gründen von Bedeutung. Erstens unterliegen Kundendaten in der Europäischen Union der Datenschutz-Grundverordnung (DSGVO), die strenge Anforderungen an die Verarbeitung, Speicherung und Weitergabe personenbezogener Daten stellt. Werden diese Daten an Dienste übermittelt, ohne dass eine entsprechende rechtliche Grundlage vorliegt, ist dies ein Datenschutzverstoß, der für das Unternehmen erhebliche rechtliche und finanzielle Konsequenzen haben kann. Zweitens ist das Vertrauen der Kunden in die Plattform von Hello Again sehr wichtig; jede Form von Databreach würde das Vertrauen beschädigen zwischen Kunde und Firma. Für uns heißt das, dass wir darauf achten

mussten, bei der Nutzung von KI-Tools ausschließlich selbst erstellte Beispieldaten zu verwenden. Keine echten Kundendaten!

4.8.3 Erlaubte KI-Tools: Firmeninterne Lösungen

Hello Again setzt auf firmenintern verwaltete KI-Lösungen. Im Rahmen der Diplomarbeit durften waren zwei Tools zur Nutzung freigegeben. Google Gemini (bezahlte Firmenversion): Hello Again verwendet die kostenpflichtige, unternehmensseitig verwaltete Version von Google Gemini. Der Unterschied zur kostenlosen öffentlichen Version ist, dass Google mit Enterprise-Verträgen garantiert, dass die Daten nicht für das Training von Modellen zu verwendet werden. Claude als Konsolentool (bezahlte Version im Firmen-Environment): Zusätzlich duften wir Claude von Anthropic verwenden, jedoch ausschließlich in der bezahlten Version. Claude wurde dabei als Konsolentool eingesetzt, also für Debugging-Unterstützung.

Hier kann man erkennen, dass die Unternehmen die neuartigen KI-Tools nicht ablehnen sondern ihre Mitarbeiter ermutigen diese sinnvoll einzusetzen.

4.8.4 Sensibilisierung für Phishing-Angriffe

Ein weiterer Bestandteil der Sicherheitseinschulung war die Aufklärung über Phishing-Angriffe. Phishing bezeichnet den Versuch, Mitarbeiter durch gefälschte E-Mails, Links oder Webseiten dazu zu bringen, sensible Informationen preiszugeben zum Beispiel Passwörter, Zugangsdaten uvm. Für Unternehmen sind Phishing-Angriffe besonders gefährlich, weil sie häufig gezielt auf bestimmte Organisationen abgestimmt sind. Wir wurden gebeten, bei jeder E-Mail mit unbekanntem Absender oder ungewöhnlichem Inhalt besonders Vorsicht zu sein, keine Links einfach so zu öffnen und Rücksprache mit unserem Betreuer zu halten.

4.8.5 Branches

Kein direkter Zugriff auf den Main Branch: Wir arbeiteten nicht direkt auf dem Main Branch des Projekts. Der Main Branch enthält den Code der das Dashboard am laufen hält. Direkte Änderungen an diesem Branch durch externe würden ein Risiko darstellen, da fehlerhafte Commits Auswirkungen auf das laufende Dashboard haben

könnten bzw haben. Eigener Feature-Branch: health-monitoring: Stattdessen wurde uns ein Branch namens health-monitoring bereitgestellt. Dieser Branch war unser main Arbeitsbereich. Er war vom Main Branch isoliert, sodass unsere Änderungen restliche Projekt nicht beeinflussen konnten. Selbst angelegte Sub-Branches: Innerhalb unseres health-monitoring-Branches legten wir uns zwei weitere Branches an, um eine klare Trennung zwischen Frontend und Backend zu gewährleisten: einen frontend-Branch für alle Arbeiten an UI und UX, sowie einen backend-Branch für die serverseitige Logik, API-Anbindungen und Datenbankoperationen. So konnten effizient Merge-Konflikte reduziert werden. Merge in das offizielle Projekt: Am Ende der Diplomarbeit wurde unser erarbeiteter Code nach der Überprüfung, durch die Devs in das offizielle Projekt gemerged.

4.8.6 Kein separates Mockup-Projekt

In vielen Firmen ist es üblich, Praktikanten oder Schülerinnen und Schülern ein Demoprojekt zur Verfügung zu stellen, an dem sie arbeiten können. In unserem Fall war dies firmenintern nicht machbar. Der Grund war personeller und zeitlicher Aufwand. Das Aufsetzen eines funktionsfähigen Mockup-Projekts hätte viel Zeit und Entwickler gekostet. Da diese Devs nicht verfügbar waren, entschied man sich für einen eigenen Branch. Diese Entscheidung hatte sowohl Vor- als auch Nachteile. Der offensichtliche Vorteil liegt darin, dass wir mit echten Codestrukturen gearbeitet haben aber der Nachteil besteht darin, dass die Fehlertoleranz geringer ist. Schwerwiegende Fehler im Branch hätten im schlimmsten Fall den Merge-Prozess erschwert. Da aber die Entwickler unsere Commits regelmäßig begutachtet haben, hatten wir diese Probleme nicht.

5 Zusammenfassung

Aufzählungen:

- Itemize Level 1
  - Itemize Level 2
    - Itemize Level 3 (vermeiden)

1. Enumerate Level 1
   a. Enumerate Level 2
      i. Enumerate Level 3 (vermeiden)

Desc Level 1

Desc Level 2 (vermeiden)

Desc Level 3 (vermeiden)

Literaturverzeichnis

[1] P. Rechenberg, G. Pomberger *et al.*, *Informatik Handbuch*, 4. Aufl. München – Wien: Hanser Verlag, 2006.

[2] Association for Progressive Communications, „Wireless technology is irreplaceable for providing access in remote and scarcely populated regions," 2006, letzter Zugriff am 23.05.2021. Online verfügbar: http://www.apc.org/en/news/strategic/world/wireless-technology-irreplaceable-providing-access

[3] Python Software Foundation, „Python 3 Documentation," 2024, letzter Zugriff am 24.02.2026. Online verfügbar: https://docs.python.org/3/

[4] G. Van Rossum und F. Drake, „Python Reference Manual," *Python Software Foundation*, 2009, letzter Zugriff am 24.02.2026.

[5] S. Ramírez, „FastAPI Documentation," 2024, letzter Zugriff am 14.03.2026. Online verfügbar: https://fastapi.tiangolo.com/

[6] ——, „FastAPI Benchmarks and Performance," *FastAPI Documentation*, 2024, letzter Zugriff am 14.03.2026. Online verfügbar: https://fastapi.tiangolo.com/benchmarks/

[7] Encode OSS, „Uvicorn Documentation," 2024, letzter Zugriff am 14.03.2026. Online verfügbar: https://www.uvicorn.org/

[8] Django Software Foundation, „ASGI Specification," 2023, letzter Zugriff am 14.03.2026. Online verfügbar: https://asgi.readthedocs.io/

[9] Pydantic Developers, „Pydantic Documentation," 2024, letzter Zugriff am 14.03.2026. Online verfügbar: https://docs.pydantic.dev/

[10] S. Colvin, „Data Validation and Settings Management using Python Type Annotations," *Pydantic Documentation*, 2024, letzter Zugriff am 14.03.2026. Online verfügbar: https://docs.pydantic.dev/latest/concepts/models/

[11] Google, „BigQuery Documentation," 2024, letzter Zugriff am 31.03.2026. Online verfügbar: https://cloud.google.com/bigquery/docs

[12] S. Melnik *et al.*, „Dremel: Interactive Analysis of Web-Scale Datasets," *Proceedings of the VLDB Endowment*, 2010, letzter Zugriff am 31.03.2026. Online verfügbar: https://research.google/pubs/pub36632/

[13] Google, „Google Cloud Documentation," 2024, letzter Zugriff am 31.03.2026. Online verfügbar: https://cloud.google.com/docs

[14] P. Mell und T. Grance, „The NIST Definition of Cloud Computing," *National Institute of Standards and Technology*, 2011, letzter Zugriff am 31.03.2026. Online verfügbar: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf

[15] ISO, „ISO/IEC 9075 SQL Standard," 2016, letzter Zugriff am 31.03.2026. Online verfügbar: https://www.iso.org/standard/76583.html

[16] D. Chamberlin und R. Boyce, „SEQUEL: A Structured English Query Language," *Proceedings of ACM SIGFIDET*, 1974, letzter Zugriff am 31.03.2026.

[17] Asana Inc., „Asana Developer Documentation," 2024, letzter Zugriff am 31.03.2026. Online verfügbar: https://developers.asana.com/docs

[18] HubSpot Inc., „HubSpot API Documentation," 2024, letzter Zugriff am 31.03.2026. Online verfügbar: https://developers.hubspot.com/docs/api/overview

[19] Google, „Google Play Developer API," 2024, letzter Zugriff am 01.04.2026. Online verfügbar: https://developers.google.com/android-publisher

[20] Apple Inc., „App Store Connect API," 2024, letzter Zugriff am 01.04.2026. Online verfügbar: https://developer.apple.com/documentation/appstoreconnectapi

Abbildungsverzeichnis

Tabellenverzeichnis

Quellcodeverzeichnis

Anhang